FUNDAMENTALS OF PROGRAMMING

Lecture 11: MULTIDIMENSIONAL ARRAYS

Instructor: Dr Aqib Perwaiz

DEPARTMENT OF COMPUTER ENGINEERING
COLLEGE OF E&ME

Multi Dimensional Arrays (Matrices)

- An array can have two or more dimensions. This permits them to model multidimensional objects, such as graph paper or the computer display screen.

- They are often used to represent **tables of values** consisting of information arranged in **rows** and **columns.**

Multi Dimensional Arrays (Matrices)

- To identify a particular element, we must specify two subscripts. By convention,

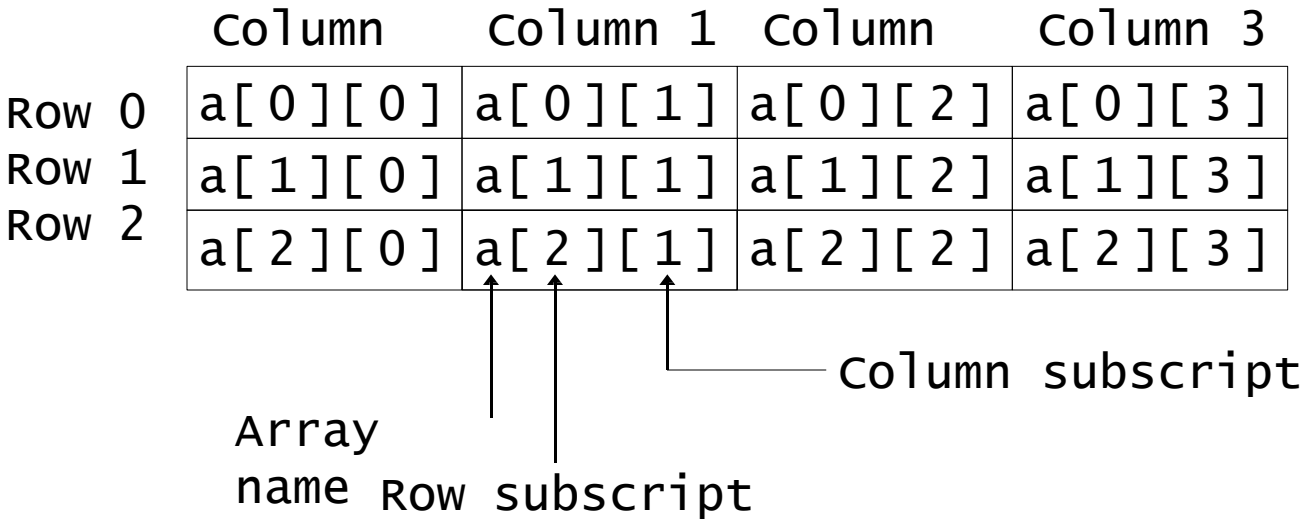
array_name[row_number][col_number]

  - For example,

arr[i][j]

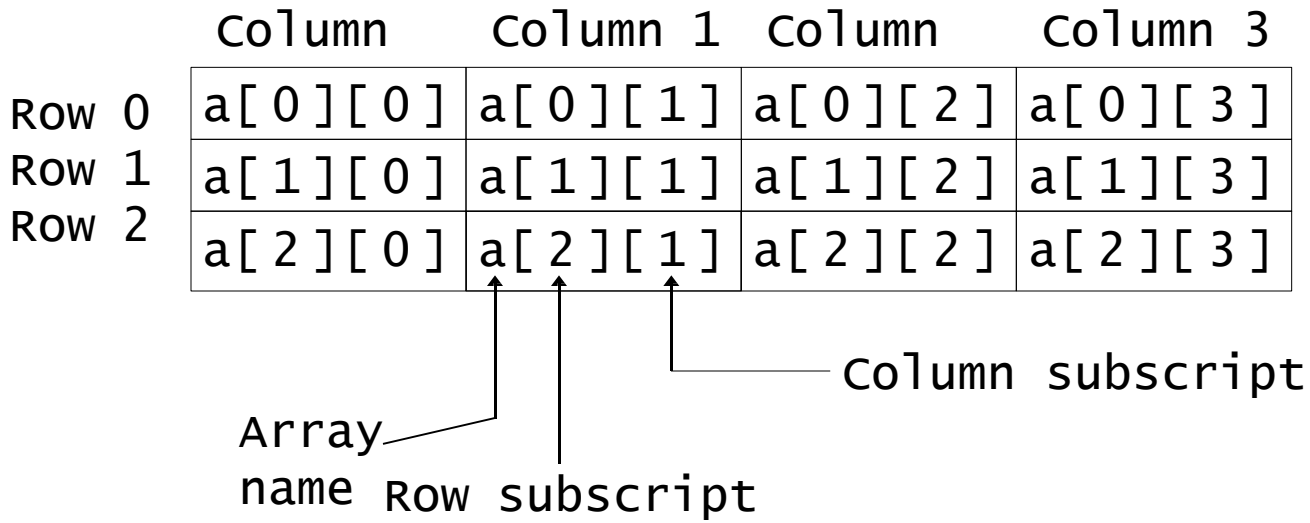
Multiple-Subscripted Arrays

- Multiple subscripted arrays
  - Tables with rows and columns (`m` by `n` array)
  - Like matrices: specify row, then column



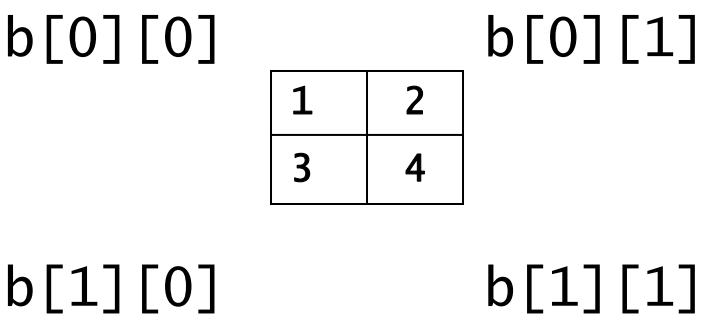
6.9 Multiple-Subscripted Arrays

- `int a[3][4];`



6.9 Multiple-Subscripted Arrays

- Initialization
  - `int b[ 2 ][ 2 ] = { { 1, 2 }, { 3, 4 } };`
  - Initializers grouped by row in braces



6.9 Multiple-Subscripted Arrays

- Initialization
  - `int b[ 2 ][ 2 ] = { { 1, 2 }, { 3, 4 } };`
  - Initializers grouped by row in braces

1	2
3	4

  - If not enough, unspecified - **elements set to** zero

    `int b[ 2 ][ 2 ] = { { 1 }, { 3, 4 } };`

1	0
3	4

- Referencing elements
  - Specify row, then column

    `cout<<b[ 0 ][ 1 ];`

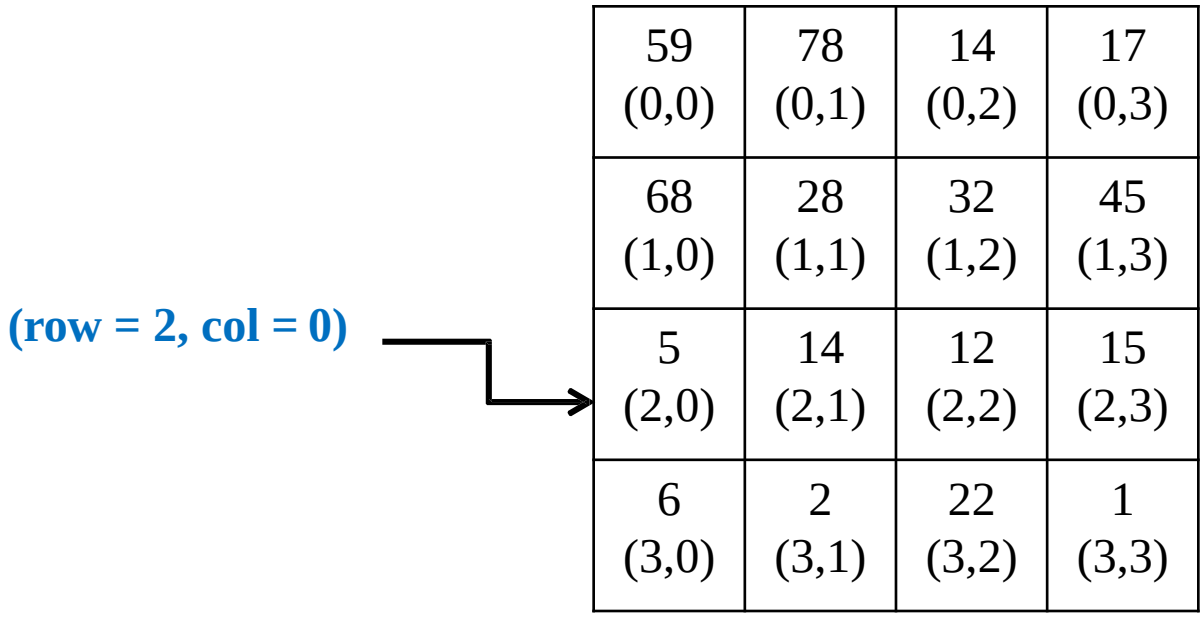
Initializing a 2D Array

- int matrix [10] [2] = { {59 , 78},{14 , 17},{68 , 28},{32 , 45},
  { 5 , 14},{12 , 15}, {6, 2}, { 22,1},
  {14,16},{2, 58} };

•The values used to initialize an array are **separated by commas and surrounded by braces**.

2-D Array (Matrix)

```
int matrix [4] [4] = { {59, 78,14, 17},{68, 28,32, 45},
                       {5, 14,12, 15} ,{6, 2, 22, 1} };
```



4 x 4 Matrix

Reading a 2D Array

```
int matrix [4][4]={{59,78,14,17},{68,28,32,45},
          {5, 14,12,15},{6,2, 22, 1}};

for(int i=0;i<4;i++)
{
for(int j=0;j<4;j++)
{
cout<<matrix[i][j]<<"\t";
}
cout<<endl;
}
```

Example

- Write a matrix with following entries

12	12
54	34
16	3
12	6
43	23
1	2

2D String

1001	A	l	i	\0						
1011	A	b	b	a	s	\0				
1021	A	l	i	n	a	\0				
1031	A	y	e	s	h	a	\0			
1041	B	a	n	o	\0					

2D array of characters

```
1  /* Fig. 6.21: fig06_21.c
2     Initializing multidimensional arrays */
3  #include <stdio.h>
4
5  void printArray( const int a[][ 3 ] ); /* function          */
   prototype
6
7                  /* function main begins program execution */
8  int main()
9  {
10    /* initialize array1, array2, array3 */
11    int   array1[  2 ][  3 ] = { { 1, 2, 3 }, { 4, 5,      6 } };
12    int   array2[  2 ][  3 ] = { 1, 2, 3, 4, 5 };
13    int   array3[  2 ][  3 ] = { { 1, 2 }, { 4 } };
14
15    cout<<"Values in        by row are:\n" ;
      array1
16    printArray( array1 );
17
18    cout<<"Values in        by row are:\n" ;
      array2
19    printArray( array2 );
20
21    cout<<"Values in        by row are:\n" ;
      array3
22    printArray( array3 );
24
23    return 0; /* indicates successful termination */
25
26 } /* end main */
27
```

```
28 /* function to output array with two rows and three columns */
29 void printArray( const int a[][ 3 ] )
30 {
31    int i; /* counter */
32    int j; /* counter */
33
34    /* loop through rows */
35    for ( i = 0; i <= 1; i++ ) {
36
37       /* output column values */
38       for ( j = 0; j <= 2; j++ ) {
39          cout<<a[ i ][ j ] ;
40       } /* end inner for */
41
42       cout<<"\n" ; /* start new line of output */
43    } /* end outer for */
44
45 } /* end function printArray */
```

Program Output

```
Values in array1 by row are:
1 2 3
4 5 6
Values in array2 by row are:
1 2 3
4 5 0
Values in array3 by row are:
1 2 0
4 0 0
```

Examples Using Arrays

- Character arrays

Its is used in programming for storing and manipulating text, such as words, names and sentences.

– Character arrays can be initialized using string literals

```
char string1[] = "first";
```

- Null character `'\0'` terminates strings
- `string1` actually has 6 elements

– It is equivalent to

```
char string1[] = { 'f', 'i', 'r', 's', 't', '\0' };
```

– Can access individual characters

`string1[ 3 ] is character ‘s’`

Character arrays

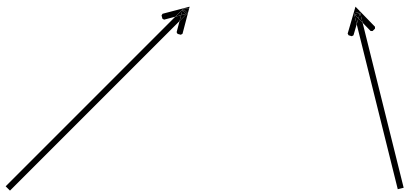
- Like all other variables and arrays

```
char name[5];  cin >> name;
cout<< "you entered " << name;
```

- The << operator displays the output until it encounters a null character.
- In case of a string variable, the cin operator places the null automatically.

Multi-Dimensional Strings (Array of Strings)

- char stu_lists [5] [10] = {“Ali”,



“Abbas”,
“Alina”,
“Ayesha”,
“Bano”,
};

Passing Elements of an Array to a Function

- Arrays are passed
  - By Reference

Passing Arrays to Functions

- Passing arrays
  - To pass an array argument to a function, specify the name of the array without any brackets

```
int myArray[ 24 ];  myFunction( myArray, 24 );
```

    - Array size usually passed to function
  - Arrays passed **call-by-reference**
  - Name of array is address of first element
  - Function knows where the array is stored
    - Modifies original memory locations
- Passing array elements
  - Passed by call-by-value
  - Pass subscripted name (i.e., `myArray[ 3 ]`) to function

Passing Arrays to Functions

- Function prototype

```
void modifyArray( int b[], int arraySize );
```

  - Parameter names optional in prototype only
    - `int b[]` could be written `int []`
    - `int arraySize` could be simply `int`

  - The following are equivalent prototypes:

```
void modifyArray( int b[], int arraySize );
void modifyArray( int [], int );
```

Acknowledgements

1. Deitel and Deitel: C++ How to Program, 7th Edition, Prentice Hall Publications
2. Robert Lafore: Object-Oriented Programming in C++, Fourth Edition, December 2001,Sams Publishing .
3. A Structured Programing Approach Using C++ by Behrouz A. Forouzan
4. www.cplusplus.com

Material in these slides has been taken from, the following resources